SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

MTD215

Examination - July Semester 2024

Application of C++ in Multimedia

Tuesday, 12 November 2024 04:00 pm - 06:00 pm

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **FOUR (4)** questions and comprises **SEVEN (7)** printed pages (including cover page).
2. You must answer **ALL** questions.
3. All answers must be written in the answer book.
4. This is **Open-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

MTD215 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Apply C++ language to program a Speedy Car Game. Implement a C++ file named player.cpp that has a car class.

(a) Create a C++ class called Car that contains as follows:

i. A string representing the username of the driver
ii. A string representing the model of the car
iii. A float number representing the max speed of the car
iv. A float number representing the current speed of the car with initial value of 0
v. An integer representing the available number of using nitro to speed up, i.e., nitro level, with initial value of 3
vi. An integer representing the car's max number of using nitro to speed up, i.e., nitro capacity, with initial value of 3.
vii. A float number representing the acceleration of the car with initial value of 2.

(8 marks)

(b) Create a Constructor of this class with initialisation of the aforementioned items.

(10 marks)

(c) Create a C++ function to carry out the following method in the C++ class.

The function `accelerate(float time)` is used to increase the current speed of the car using first equation of motion in classical mechanics (i.e., $v = v_0 + a \cdot t$). Do note that 1) if **`nitroLevel`** is still above zero, the acceleration will be doubled to increase the speed, otherwise the acceleration will be same as its set value; 2) Stop accelerating if the car reaches the maximum speed.

Comment in the key line of code to explain and elaborate the functions and their rationale.

(7 marks)

Question 2

(a) Based on what you have learnt in C++, explain the **FOUR (4)** major principles/characteristics of OOP. Write a few lines of C++ codes to reflect the principles/characteristics for illustration, respectively.

Principles/characteristics of OOP	Explanation	Example Code

(12 marks)

(b) Taking the input from user is a key task in SDL, discuss the typical coding blocks (pseudo-code) of SDL event-driven program and its handler.

(8 marks)

(c) The original SDL library can only accept bmp image files. Provide and discuss the solution in **FIVE (5) steps** if we want our SDL program to use other image formats, such as png file.

Step 1	
Step 2	
Step 3	
Step 4	
Step 5	

(5 marks)

Question 3

Identify suitable coding modules below.

```
1   #include <iostream>
2   using namespace std;
3
4   class Rectangle {
5   public:
6
7       //To comment
8       Rectangle() {
9           width = 5;
10          height = 5;
11          cout << "a default rectangle created..." << endl;
12      }
13
14      //To comment
15      Rectangle(int nx, int ny) {
16          width = Nx; height = ny;
17      }
18
19      //To comment
20      ~Rectangle() {}
21
22      //To comment
23      Rectangle operator+ (const Rectangle p1) {
24
25          return Rectangle(width + p1.width, height + p1.height);
26          cout << "Performing sum up of two rectangles..." << endl;
27      }
28
29      int getWidth() { return width; }
30
31      int getHeight() { return height; }
32
33  private:
34      int width;
35      int height;
36  };
37
38  int main(int argc, char* argv[]) {
39      cout << "Hello World" << endl;
40
41      //To comment
42      Rectangle ptA();
43      Rectangle ptB(3, 4)
44
45      Rectangle ptC = ptA + ptB;
46      cout << "The sum of RectangleA and RectangleB is:= (" << ptC.getWidth()
47          << "," << ptC.getHeight() << ")" << endl;
48
49      return 0;
50  }
```

(a) Examine the code and explain them as comments at Line 7, 14, 19, 22 and 41. (10 marks)

(b) There are **FIVE (5)** errors/bugs hidden in the code. Identify and write the solution to fix the errors for each of them.

Error/Bug	Solution
Line number ? what error?	
Line number ? what error?	
Line number ? what error?	
Line number ? what error?	
Line number ? what error?	

(10 marks)

(c) Imagine you are the user, and you execute the above program, write down the corresponding output of the program in the correct order.

(5 marks)

Question 4

Appraise the use of SDL for geometry rendering. SDL has primitive functions for drawing rectangles. Write **FOUR (4)** functions for Question 4a, 4b, 4c and 4d based on the SDL primitive functions and fundamental C++ implementations.

```
#include <SDL.h>
#include <cmath>
#include <iostream>

const int WINDOW_WIDTH = 800;
const int WINDOW_HEIGHT = 600;
const int RECT_GAP = 100;
const int COLOR_CHANGE_SPEED = 1; // Varying increment of changing RGB value

void interface(int* length, int* width) {
    //Your method of Q4(a) goes here

}

void draw_rect2(SDL_Renderer* renderer, SDL_Rect* rect2) {
    //Your method of Q4(b) goes here

}

void draw_rect1(SDL_Renderer* renderer, SDL_Rect* rect1,int r, int g, int b) {
    //Your method of Q4(c) goes here
}

void change_color(Uint8* r, Uint8* g, Uint8* b) {
    //Your method of Q4(d) goes here

}

int main(int argc, char* argv[]) {
    SDL_Init(SDL_INIT_VIDEO);

    int rec1_length;
    int rec1_width;


    SDL_Window* window = SDL_CreateWindow("Color Changing Rectangle",
        SDL_WINDOWPOS_CENTERED,
        SDL_WINDOWPOS_CENTERED,
        WINDOW_WIDTH, WINDOW_HEIGHT,
        SDL_WINDOW_SHOWN);

    interface(&rec1_length, &rec1_width);

    int rec2_length = rec1_length + RECT_GAP*2;
    int rec2_width = rec1_width + RECT_GAP * 2;;

    SDL_Renderer* renderer = SDL_CreateRenderer(window, -1, SDL_RENDERER_ACCELERATED);

    SDL_Rect rect1 = { WINDOW_WIDTH / 2 - rec1_width / 2, WINDOW_HEIGHT / 2
    - rec1_length / 2, rec1_width, rec1_length };
    SDL_Rect rect2 = { WINDOW_WIDTH / 2 - rec2_width / 2, WINDOW_HEIGHT / 2
    - rec2_length / 2, rec2_width, rec2_length };

    Uint8 r = 0, g = 255, b = 0;
    SDL_bool running = SDL_TRUE;
    SDL_Event event;

    draw_rect2(renderer, &rect2);

    while (running) {
        while (SDL_PollEvent(&event)) {
            if (event.type == SDL_QUIT || (event.type == SDL_KEYDOWN &&
            event.key.keysym.sym == SDLK_ESCAPE)) {
                running = SDL_FALSE;
            }
        }

        change_color(&r, &g, &b);

        draw_rect1(renderer, &rect1, r, g, b);

    }

    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();

    return 0;
}
```

The SDL program above draws one small rectangle and one big rectangle with same centers but different sizes defined by user inputs and the variable `RECT_GAP`. Moreover, the small rectangle will change the color itself over time (i.e., frame by frame). The color changing can be a fixed increment of current RGB values or a continuous random setting.

Write the implementation for `interface`, `draw_rect1`, `draw_rect2` and `change_color` function to produce each of following output respectively:

(a) Write the implementation for `interface` function to obtain the user input of width and length of smaller rectangle. The user interface is illustrated as below:

```
Please input the length of rectangle in unit of pixels
250
Please input the width of rectangle in unit of pixels
400
```

(4 marks)

(b) Write the implementation for `draw_rect2` function to draw the large green rectangle using user inputs and the variable `RECT_GAP`.

(10 marks)

(c) Write the implementation for `draw_rect1` function to draw the small rectangle using RGB parameters. Do note to apply delay function for controlling speed of refreshing frame.

(6 marks)

(d) Write the implementation for `change_color` function to update the RGB values. Do note to apply variable of `COLOR_CHANGE_SPEED` for controlling increments of changing RGB values.

(5 marks)

----- END OF PAPER -----